



CS343: Operating Systems
Quiz-1, August 29, 2024
Indian Institute of Technology Guwahati

Section A: Objective Types

Marks: 1 x 10 = 10 marks

Q1. Which of the following is/are dynamically allocated in the memory: -

- a. Compile code b. Global variables c. Stack d. Heap

Ans. (c) and (d)

Q2. Invoking the exec system call will change which information in the PCB entries of a process?

- a. Process identifier (PID)
b. Page table entries
c. The value of the program counter stored within the user space context on the kernel stack

Ans. (a) does not change. (b) and (c) change because the process gets a new memory image (and hence new page table entries pointing to the new image)

Q3. Which of the following is same for the parent and child processes immediately after invoking the fork system call?

- a. The process identifier. b. The contents of the file descriptor table.

Ans. (a) is not identical, as every process has its own unique PID in the system.

(b) is identical, as the child gets an exact copy of the parent's file descriptor table.

Q4. During trap handling an x86 instruction "int n" is executed. Which of the following statements is/are true regarding this execution.

- a. This instruction is always invoked by privileged OS code.
b. This instruction causes the CPU to set its EIP to address value n.
c. This instruction causes the CPU to lookup the Interrupt Descriptor Table (IDT) using the value n as an index.
d. This instruction is always executed by the CPU only in response to interrupts from external hardware, and never due to any code executed by the user.

Ans. c

Q5. Which one or more of the following CPU scheduling algorithms can potentially cause starvation?

- a. First Come First Serve b. Round Robin c. Priority Scheduling d. Shortest Job First

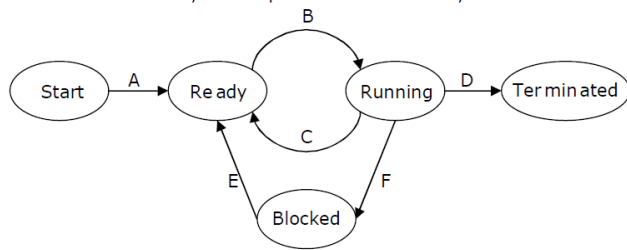
Ans. c and d

Q6. Using the same process state diagram as in Q5. Which of the following statements is/are true?

- a. A running process can move to ready state.
b. A ready process can move to running state.
c. A blocked process can move to running state.
d. A blocked process can move to ready state.

Ans. a, b, and d

Q7. In the following process state transition diagram for a uniprocessor system, assume that there are always some processes in the ready state: Now consider the following statements:



- If a process makes a transition D, it would result in another process making transition A immediately.
- A process P2 in blocked state can make transition E while another process P1 is in running state.
- The OS uses preemptive scheduling.
- The OS uses non-preemptive scheduling.

Which one or more of the above statements are true?

Ans. b and c

Q8. In xv6, state the system call(s) that result in new struct proc objects being allocated.

Ans: fork

Q9. Consider the context switch of a CPU from the context of process P1 to that of process P2 in xv6. Consider the following two events in the chronological order of the events during the context switch: (E1) the ESP (stack pointer) shifts from pointing to the kernel stack of P1 to the kernel stack of P2; (E2) the EIP (program counter) shifts from pointing to an address in the memory allocated to P1 to an address in the memory allocated to P2. Which of the following statements is/are true regarding the relative ordering of events E1 and E2?

- E1 occurs before E2.
- E2 occurs before E1.
- E1 and E2 occur simultaneously via an atomic hardware instruction.
- The relative ordering of E1 and E2 can vary from one context switch to the other.

Ans. a

Q10. Consider the following actions that happen during a context switch from thread/process P1 to thread/process P2 in xv6. (One of P1 or P2 could be the scheduler thread as well.) Arrange the actions below in chronological order, from earliest to latest.

- Pop the callee-save registers from the kernel stack of P2
- Push the callee-save registers onto the kernel stack of P1
- Switch ESP from kernel stack of P1 to that of P2
- Push the EIP where execution of P1 stops onto the kernel stack of P1.

Ans: dbca

Section B

2 x 5 marks= 10 marks

Q11. List the key steps that happen during “exec” system call.

Ans.

- A. Copy new executable into memory**
- B. Create new stack, heap**
- C. Switch process page table to use new memory image**
- D. Process begins to run new code after system call ends**

Q12. Consider process P invoking a system call on xv6. During trap handling, the CPU performs certain steps as part of the “int n” instruction. What does it save on the kernel stack of process P, and why is this information saved?

Ans. On kernel stack, save old esp, eip (where execution stopped before interrupt occurred, so that it can be resumed later)

Q13. A follow up question after Q12. After completing the execution of the system call, which instruction is responsible for returning to process P, and what does it do?

Ans. Return from trap instruction: “iret” does the opposite of int

- a. Pop values pushed by “int”**
- b. Change back privilege level**

Q14. Consider two processes, P1 and P2. P1 is currently executing on the CPU, and after some time, P2 needs to execute. To perform this context switching in xv6, the “swtch” function is invoked. Comment on what the “swtch” function does to the “callee-saved” registers of P1 and the “esp” register.

Ans.

- a. Push remaining registers on old kernel stack (only callee save registers need to be saved)**
- b. Switch esp from old kernel stack to new kernel stack**

Q15. Consider three processes PL, PM, and PH with priority PH>PM>PL executing on a uniprocessor using preemptive priority scheduling. Can there be a case where PM can preempt PL, but PH still cannot preempt PL at a certain point of time, thus this may cause PH to wait infinitely? If yes how can this be rectified, if no justify with one statement.

Ans. Yes, Priority inversion is a bug that occurs when a high priority task is indirectly preempted by a low priority task. For example, the low priority task holds a mutex that the high priority task must wait to continue executing. Process L should, in fact, be temporarily of higher priority than process M, on behalf of process H.

Section C

6 marks + 4 marks = 10 marks

Q16. Consider 4 processes P, Q, R, S that were to be scheduled on a CPU using a dynamic round robin scheduling with a time quantum of $2x$ clock cycles, where x indicates the round number starting with 1, and then 2, and then 3, and so on for subsequent rounds, i.e. time quantum increases in each round. Each process will get at most one slot of running state in each round. The arrival time of P, Q, R and S are at clock cycles 0, 1, 2 and 3, respectively. It was found that P and R completed execution at clock cycles 12 and 18, respectively. The turnaround time of S is two clock cycles more than the turnaround time of Q. If the average turnaround time of the 4 processes is 28 cycles, with the help of a neat Gantt chart answer the following.

- How long is the CPU burst of each of the processes?
- How many preemptive contexts switching happened overall and mention the clock cycles too?
- What is the waiting time for process R?

Ans.

Dynamic round robin scheduling, $T = 2x$ clock cycles, where $x=1,2,3...$

Each process will get at most one slot of running state in each round.

Arrival $P=0$, $Q=1$, $R=2$ and $S=3$

P and R completed execution at clock cycles 12 and 18, respectively.

Turnaround time of S is two clock cycles more than the turnaround time of Q.

Average turnaround time of the 4 processes is 28 cycles.

$TAT = 12 + y + (18-2) + (y+2) = 28 \times 4 \Rightarrow y = 41$, i.e. TAT of Q = 41

So, Completion time of Q = TAT of Q + Arrival of Q = $41+1 = 42$

Thus, completion time of S = TAT of S + Arrival of S = $(41+2)+3=46$

	Round 1				Round 2				Round 3		Round 4	
	P	Q	R	S	P	Q	R	S	Q	S	Q	S
0	2	4	6	8	12	16	18	22	28	34	42	46

TAT of:-

$P = 12-0 = 12$,

$Q = 42-1 = 41$,

$R = 18-2 = 16$

$S = 46-3 = 43$

Average TAT = $(12+41+16+43)/4=112$

(a) CPU burst Time

P = 6

Q = 20

R = 4

S = 16

(b) Context switching – 8 times @clock cycles 2,4,6,8,16,22,28,34

(c) Waiting time of R = TAT – BT = $16 - 4 = 12$

Q17. Consider the following four processes with arrival times (in milliseconds) and their length of CPU bursts (in milliseconds) as shown below:

Process	P1	P2	P3	P4
Arrival Time	0	1	3	4
Burst Time	3	1	3	Z

These processes are run on a single processor using preemptive Shortest Remaining Time First scheduling algorithm. If the average waiting time of the processes is 1 millisecond, then the value of Z is ____.

Ans. Solving by trial and error

Let Z = 1

Av waiting time = $\{(4-0-3) + (2-1-1) + (8-3-3) + (5-4-1)\} / 4 = 0.75$

Let Z=2

= $\{(4-0-3) + (2-1-1) + (9-3-3) + (6-4-2)\} / 4 = 1$

Thus, Z=2